Лабораторная работа №6

Тема: Защищённые страницы и навигация по ролям во frontend

Цель: Реализовать protected pages и route guards с использованием существующего authState, не дублируя ЛР-5.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте существующий проект leson8prisma из лабораторной №5. Файлы клиентской части находятся в каталоге frontend/ в корне проекта; браузер открывает приложение по адресу http://localhost:3000/. Сервер Express уже настроен на отдачу этой страницы.

Файлы authState.js, api.js с apiFetch(), форма login и logout уже разработаны в ЛР-5. Продолжайте их дорабатывать; не создавайте новую папку static/, не переносите файлы и не запускайте отдельный веб-сервер на порту 5500.

> **Важно**
>
> В этой работе проверка страниц выполняется во frontend только для удобства пользователя. Доступ к API по-прежнему защищается middleware на backend.

1. Проверьте существующие файлы

```
frontend/
  index.html
  app.js
  api.js
  authState.js
  config.js
  styles.css
```

Файлы app.js, api.js и authState.js используйте из ЛР-5. Не перезаписывайте работающий проект полностью.

2. Проверьте authState.js

Функция getCurrentUser() должна возвращать объект пользователя после login или null до входа. Функции setAuth() и clearAuth() уже созданы в ЛР-5.

```
import { getCurrentUser } from "./authState.js";

console.log(getCurrentUser()); // null до login
```

3. Добавьте меню и дополнительные секции

В существующий frontend/index.html добавьте ссылки навигации и новые секции. Не удаляйте готовую форму входа из ЛР-5: она расположена в секции с id="guest-section". Секция id="user-section" также уже существует.

```
<nav>
  <a id="catalog-link" href="#/catalog">Каталог</a>
  <a id="login-link" href="#/login">Войти</a>
  <a id="profile-link" href="#/profile" hidden>Профиль</a>
  <a id="admin-link" href="#/admin" hidden>Панель admin</a>
```

```
</nav>

<section id="catalog-section">Каталог товаров</section>
<section id="profile-section" hidden>
  <h2>Профиль</h2><p id="profile-name"></p>
</section>
<section id="admin-section" hidden>
  <h2>Панель администратора</h2>
  <p>Создание категорий и товаров — только admin.</p>
</section>
```

О форме входа

В ЛР-5 используются идентификаторы guest-section и user-section. Сохраните их: добавьте новые разделы отдельно и используйте те же id в JavaScript. Примеры ниже опираются на уже существующие элементы.

4. Подключите элементы в app.js

К существующему frontend/app.js добавьте ссылки на новые элементы. Если переменные с теми же именами уже объявлены, повторно их не создавайте.

```
import { getCurrentUser } from "./authState.js";

const catalogSection = document.querySelector("#catalog-section");
const profileSection = document.querySelector("#profile-section");
const adminSection = document.querySelector("#admin-section");
const loginLink = document.querySelector("#login-link");
const profileLink = document.querySelector("#profile-link");
const adminLink = document.querySelector("#admin-link");
const profileName = document.querySelector("#profile-name");
```

5. Реализуйте requireUser()

```
function requireUser() {
  if (!getCurrentUser()) {
    location.hash = "#/login";
    return false;
  }
  return true;
}
```

Если пользователь не вошёл, браузер переключается к форме входа. Функция возвращает false, чтобы отменить показ защищённого раздела.

6. Реализуйте requireAdmin()

```
function requireAdmin() {
  const user = getCurrentUser();
  if (!user) {
    location.hash = "#/login";
    return false;
  }
  if (user.role !== "admin") {
```

```
    alert("Недостаточно прав");
    location.hash = "#/catalog";
    return false;
  }
  return true;
}
```

Для обычного пользователя выводится сообщение и открывается каталог. Серверный requireRole("admin") сохраняется без изменений.

7. Реализуйте показ разделов

В данном примере каталог и форма входа — публичные разделы; профиль и admin — защищённые. Используем уже существующую guestSection из ЛР-5.

```
function showOnly(section) {
  for (const item of [guestSection, userSection, catalogSection, profileSection,
adminSection]) {
    item.hidden = item !== section;
  }
}

function renderPage() {
  const page = location.hash || "#/catalog";

  if (page === "#/profile") {
    if (!requireUser()) return;
    profileName.textContent = getCurrentUser().name;
    showOnly(profileSection);
  } else if (page === "#/admin") {
    if (!requireAdmin()) return;
    showOnly(adminSection);
  } else if (page === "#/login") {
    if (getCurrentUser()) {
      location.hash = "#/profile";
      return;
    }
    showOnly(guestSection);
  } else {
    showOnly(catalogSection);
  }
}
```

Учёт существующего интерфейса

В ЛР-5 также использовался userSection. Если он содержит кнопки «Мои заказы» и «Выйти», их можно перенести в profile-section или сохранить отдельно; не оставляйте два одновременно видимых раздела. При переносе проверьте ссылки querySelector на существующие элементы.

8. Добавьте навигацию по ролям

```
function renderNavigation() {
  const user = getCurrentUser();
  loginLink.hidden = Boolean(user);
  profileLink.hidden = !user;
  adminLink.hidden = user?.role !== "admin";
```

```
}
```

Вошедший пользователь видит профиль, а администратор — дополнительно ссылку «Панель admin».

9. Подключите hashchange и первый запуск

```
window.addEventListener("hashchange", renderPage);
renderNavigation();
renderPage();
```

Выполните этот код после объявления функций, когда DOM уже создан. Для script type="module" из ЛР-5 это стандартный вариант.

10. Обновите существующие обработчики login и logout

В обработчике успешного login после showAuthenticatedUI(user) вызовите обновление меню и переход на профиль:

```
renderNavigation();
location.hash = "#/profile";
```

В обработчике logout после clearAuth() вызовите:

```
renderNavigation();
location.hash = "#/login";
```

В ЛР-5 функция showAuthenticatedUI(user) открывала userSection. Теперь отображением разделов управляет renderPage(). Чтобы не было конфликтов, удалите из showAuthenticatedUI() изменение userSection.hidden и после успешного login вызовите renderNavigation() и установите location.hash = "#/profile". При выходе очищайте authState и обновляйте навигацию.

11. Проверьте работу интерфейса

Сценарий	Ожидаемый результат
Гость открывает #/catalog	Каталог виден
Гость открывает #/profile	Переход на #/login
Гость открывает #/admin	Переход на #/login
user открывает #/profile	Профиль и имя пользователя
user открывает #/admin	Сообщение «Недостаточно прав» и каталог
admin открывает #/admin	Панель администратора
После logout	Ссылка входа, защищённые страницы скрыты
После F5	Пользователь становится гостем

12. Проверьте, что backend по-прежнему защищён

Обычный пользователь может изменить URL и JavaScript в браузере. Но запрос POST /api/products с его JWT всё равно должен возвращать 403, а без JWT — 401 (если такие ограничения уже реализованы в ЛР-4).

Проверять нужно именно статус HTTP-ответа во вкладке Network, а не только видимость кнопок.

13. Самостоятельное задание

- Добавьте в меню ссылку «Мои заказы», доступную только после входа. Само получение заказов выполняйте только если маршрут GET /api/orders/my уже создан.
- Добавьте отдельную кнопку или ссылку «Категории» с публичным GET /api/categories.
- Сделайте вывод сообщения «Необходимо войти» вместо открытия профиля гостем.
- После ошибки 403 оставляйте вход действующим и показывайте понятное сообщение.
- Добавьте обработку неизвестного адреса, например #/unknown → каталог.

Подсказка для заказов

Проверка отображения: сначала getCurrentUser(), затем apiFetch("/orders/my"). На backend принадлежность заказа определяется по req.user.id. Не передавайте userId из frontend как доказательство владельца.

Требования к результату

- Frontend по-прежнему открывается через http://localhost:3000/ из существующей папки static.
- Существующие login, logout и apiFetch продолжают работать.
- Гость не видит профиль и admin-раздел.
- Обычный пользователь видит профиль, но не панель admin.
- Admin видит административный раздел.
- Защита API продолжает работать независимо от frontend.
- При обновлении страницы состояние входа сбрасывается — в текущем учебном варианте это ожидаемо.

Критерии оценивания

№	Критерий
1	Работает переключение разделов по location.hash
2	Реализована проверка requireUser()
3	Реализована проверка requireAdmin()
4	Навигация меняется после login/logout
5	Гость не открывает protected UI
6	User не открывает admin UI
7	Admin открывает admin UI
8	Backend по-прежнему возвращает 401/403

№	Критерий
9	Существующий frontend и API продолжают работать
10	Студент объясняет, почему guard не заменяет backend authorization